

Descripcion de la Persistencia de Datos

Proyecto: Tienda Retail Lumina - Arquitectura de Microservicios

Asignatura: Desarrollo FullStack III (DSY1106)

1. Enfoque general

El sistema aplica el patron **Database per Service** (una base de datos por microservicio). Cada microservicio es propietario exclusivo de su esquema y ningun otro servicio accede directamente a la base de datos de otro: la comunicacion siempre ocurre a traves de la API REST por medio del BFF Gateway.

Microservicio	Base de datos	Puerto MySQL
ms-usuarios	db_usuarios	3307
ms-productos	db_productos	3308
ms-bodega	db_bodega	3309
ms-carrito	db_carrito	3310
ms-delivery	db_delivery	3311
ms-pagos	db_pagos	3312

Cada base de datos corre en su propio contenedor **MySQL 8.0** con un volumen Docker persistente (`vol_mysql_*`), de modo que los datos sobreviven a reinicios.

2. Tecnologia de persistencia: JPA / Hibernate

La persistencia se implementa con **Spring Data JPA** sobre **Hibernate** como proveedor ORM. Cada microservicio define:

- **Entidades** (`@Entity`) mapeadas a tablas con anotaciones JPA.
- **Repositorios** (`extends JpaRepository`) que proveen el CRUD y consultas derivadas por nombre de metodo.
- **Servicios** que contienen la logica de negocio y orquestan los repositorios.

Ejemplo de entidad (Producto)

```
@Entity
@Table(name = "productos")
public class Producto {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String nombre;

    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal precio;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "marca_id")
    private Marca marca;

    private Boolean activo;
}
```

Ejemplo de repositorio

```
public interface ProductoRepository extends JpaRepository<Producto, Long> {
    List<Producto> findByActivoTrue();
    List<Producto> findByNombreContainingIgnoreCase(String nombre);
}
```

3. Configuración (application.yml)

Cada servicio configura su datasource con variables de entorno (con valores por defecto para desarrollo local):

```
spring:
  datasource:
    url: ${SPRING_DATASOURCE_URL:jdbc:mysql://localhost:3308/db_productos}
    username: ${SPRING_DATASOURCE_USERNAME:lumina_user}
```

```
password: ${SPRING_DATASOURCE_PASSWORD:lumina_pass}
driver-class-name: com.mysql.cj.jdbc.Driver
jpa:
  hibernate:
    ddl-auto: update
  properties:
    hibernate:
      dialect: org.hibernate.dialect.MySQLDialect
```

- **ddl-auto: update** : Hibernate crea/actualiza el esquema automaticamente a partir de las entidades, evitando scripts SQL manuales.
- **Externalizacion**: las credenciales y URLs se inyectan por variables de entorno (perfil `docker`), lo que permite el mismo artefacto en distintos entornos.

4. Persistencia en pruebas (H2)

Para las pruebas unitarias e integracion se usa una base de datos **H2 en memoria**, configurada en `src/test/resources/application-test.yml` :

```
spring:
  datasource:
    url: jdbc:h2:mem:testdb
    driver-class-name: org.h2.Driver
  jpa:
    hibernate:
      ddl-auto: create-drop
    properties:
      hibernate:
        dialect: org.hibernate.dialect.H2Dialect
```

Esto permite ejecutar las pruebas de repositorio (`@DataJpaTest`) sin depender de una instancia MySQL real, garantizando rapidez y aislamiento.

5. Relaciones y modelos destacados

- **ms-productos**: Producto `N...1` Marca (`@ManyToOne`).
- **ms-carrito**: Carrito `1...N` ItemCarrito (`@OneToMany`).

- **ms-bodega:** Inventario y AlertaStock (generacion automatica de alertas cuando el stock cae bajo el minimo).
- **ms-delivery:** Delivery 1...N HistorialDelivery (trazabilidad de estados).
- **ms-pagos:** Pago con enum EstadoPago .

6. Resumen

La persistencia es **poliglota a nivel de instancia** (6 bases MySQL independientes) pero **homogenea en tecnologia** (JPA/Hibernate), lo que asegura desacoplamiento entre servicios, escalabilidad independiente y consistencia en el acceso a datos dentro de cada dominio.